

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5112

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I K . B h a v a n a (1 9 2 5 2 4 1 7 8) _ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.Bhavana

Question 1: Debugging Hash Function Implementation (SHA Family)

Scenario

A Python program implementing a SHA-256-style hashing algorithm produces inconsistent hash outputs for the same input. The task requires identifying errors in message padding and block processing, fixing bitwise operation and endianness bugs, and optimizing the code for large files (≥50 MB).

Why the Symptom Occurs

SHA-256 is fully deterministic by specification (FIPS 180-4): the same input byte sequence must always produce the same 256-bit digest. When the same message hashes differently across runs, the cause is never randomness in SHA-256 itself — it is a state, ordering, or arithmetic bug in the implementation that makes the output depend on something other than the input, such as leftover state from a previous call, incorrect byte ordering during packing, or unbounded integer growth that behaves differently depending on prior operations.

Step 1 — Problem Identification

The table below lists each defect found in the original implementation, its underlying root cause, and the concrete impact it has on correctness or security.

Defect Identified	Root Cause	Impact
Shared hash-state variables	The internal hash registers (H0–H7) are declared once at module/program level instead of being freshly initialized inside every hashing call.	Calling the hash function twice on the identical input produces two different digests, because the second call starts from the first call's leftover state instead of the standard initial values.
Incorrect padding order	The mandatory padding step (append a single '1' bit, zero-pad, then append the 64-bit message length) is implemented with the length field in the wrong byte order.	The padded message no longer matches the byte layout required by the specification, so blocks are misaligned before compression even begins.
Wrong endianness for message words	32-bit message words and the final length field are packed/unpacked using little-endian byte order instead of the big-endian order SHA-256 requires.	Every word entering the compression function is byte-reversed relative to spec, so the digest is wrong even when every other step is implemented correctly.

Defect Identified	Root Cause	Impact
Unmasked 32-bit arithmetic	Python integers are unbounded, so additions inside the compression rounds are not explicitly wrapped back into the 32-bit range.	Register values silently grow beyond 32 bits, producing intermediate results that diverge from the fixed-width arithmetic the algorithm assumes, corrupting the final digest.

Step 2 — Debugging Approach

Each defect above is resolved using a systematic, targeted correction. The table maps every defect to the specific debugging action taken and the principle or standard it restores compliance with.

Defect Addressed	Debugging Action Taken	Standard / Principle Restored
Shared hash-state variables	State registers are re-initialized to the standard SHA-256 initial values inside the function body on every call, so no call can ever see another call's leftover state.	Statelessness / determinism required by FIPS 180-4
Incorrect padding order	The padding routine is rewritten to append the '1' bit, then zero-bytes until the length is congruent to 56 modulo 64, then the original bit-length encoded in the correct byte order.	FIPS 180-4 padding rule
Wrong endianness	All word packing and unpacking, including the final length suffix, is switched to big-endian throughout the padding and compression stages.	FIPS 180-4 big-endian word convention
Unmasked arithmetic	Every addition inside the compression rounds is explicitly reduced back into the 32-bit range after the operation.	Fixed-width (32-bit) modular arithmetic required by the algorithm

Step 3 — Optimization

Once correctness is restored, the implementation is optimized for the scale/performance target stated in the scenario. Each technique below targets the true bottleneck rather than an incidental inefficiency.

Optimization Technique	What It Changes	Performance Benefit
Chunked / streamed reading	Instead of loading an entire large file into memory before hashing, the file is read and processed in fixed-size chunks (e.g. a few megabytes at a time).	Memory usage stays constant regardless of file size, so a 50 MB or 500 MB file both hash with the same, low memory footprint.
Buffer-based padding construction	The final padding step is built using a mutable, appendable buffer instead of repeatedly creating new copies of the message during padding.	Avoids the quadratic-time cost that repeated copying would otherwise introduce, keeping padding cost proportional to message size.
Use of an optimized/native hashing routine for production throughput	For the large-file (≥ 50 MB) case specifically, the hashing work is delegated to a highly optimized, hardware-accelerated hashing routine rather than a manual, interpreted round-by-round computation.	Substantially reduces wall-clock hashing time for large files, since the heavy per-block computation runs at native/hardware speed instead of interpreted speed.

Step 4 — Analysis / Evaluation

Evaluation Criterion	Finding
Determinism restored	After the fix, hashing the same input twice in the same program run always yields the identical digest, matching the correctness requirement of a cryptographic hash function.
Integrity vs. basic checksums	A checksum (e.g. CRC32) is designed only to catch accidental transmission errors and is linear, so it is computationally easy to construct a different message with the same checksum. A cryptographic hash such as SHA-256 is pre-image resistant, second-pre-image resistant, and collision resistant, meaning an attacker cannot feasibly construct a different input that produces the same digest.
Avalanche effect	Because every input bit is mixed through many rounds of non-linear processing, changing a single input bit changes roughly half of the output bits. This property, absent in basic checksums, is what makes tampering with hashed data detectable.
Scalability	With streaming and constant memory usage, the implementation now scales to arbitrarily large files without a corresponding increase in memory consumption, satisfying the ≥ 50 MB requirement stated in the scenario.

Question 2: Optimization of Secure Socket Layer (SSL/TLS) Simulation

Scenario

A Python-based simulation of the SSL/TLS handshake shows delays and occasional handshake failures. The task requires debugging certificate validation and key exchange steps, optimizing the handshake to reduce latency, refactoring for session resumption, and analyzing the performance/security trade-off.

Why the Symptom Occurs

TLS handshake latency comes primarily from expensive asymmetric operations (key generation, key exchange, certificate checks) that are repeated on every connection. Handshake failures typically trace back to incomplete certificate validation or to weak/collision-prone random number generation used for keys and nonces, which can cause the protocol's state machine to reject a connection it should have accepted, or worse, accept a connection it should have rejected.

Step 1 — Problem Identification

The table below lists each defect found in the original implementation, its underlying root cause, and the concrete impact it has on correctness or security.

Defect Identified	Root Cause	Impact
Shallow certificate validation	Certificate checking verifies only the subject name and skips expiry-date checking and issuer/chain-of-trust verification.	A certificate that is expired, self-signed, or issued by an untrusted authority can pass validation, undermining the entire purpose of the TLS handshake.
Key-exchange parameters regenerated every handshake	Fresh Diffie–Hellman domain parameters (a large prime and generator) are generated from scratch on every single handshake instead of reusing a standard, pre-agreed safe group.	This is the single largest contributor to handshake latency, since generating large, verifiably safe primes is computationally expensive.
No session resumption	There is no cache of previously negotiated session keys, so every reconnect from the same client repeats the full asymmetric handshake.	Returning clients pay the full handshake cost every time, even though TLS is designed to allow a cheap resumed handshake for known clients.

Defect Identified	Root Cause	Impact
Weak randomness for keys and nonces	Private key material and handshake nonces are generated using a general-purpose, non-cryptographic random number generator.	Predictable or occasionally colliding values can cause handshake state to be rejected as invalid, explaining the reported intermittent handshake failures, and also weakens key secrecy.

Step 2 — Debugging Approach

Each defect above is resolved using a systematic, targeted correction. The table maps every defect to the specific debugging action taken and the principle or standard it restores compliance with.

Defect Addressed	Debugging Action Taken	Standard / Principle Restored
Shallow certificate validation	Certificate validation is extended to check subject identity, the validity time window (not-before / not-after), and that the issuing authority is present in a trusted certificate authority store.	Chain-of-trust validation required by the TLS certificate model
Parameters regenerated every handshake	The handshake is refactored to use a fixed, standard, publicly vetted Diffie–Hellman group, with only the small per-session private exponent generated fresh.	Standard, reusable domain parameters (as used in real TLS implementations)
No session resumption	A session cache keyed by client identity is introduced, storing a resumable session key with an expiry time so a returning client can skip the full handshake.	TLS session resumption / session ticket mechanism
Weak randomness	All key and nonce generation is switched to a cryptographically secure random number source.	Cryptographically secure randomness requirement for key material

Step 3 — Optimization

Once correctness is restored, the implementation is optimized for the scale/performance target stated in the scenario. Each technique below targets the true bottleneck rather than an incidental inefficiency.

Optimization Technique	What It Changes	Performance Benefit
Reuse of standard DH group	Removes the expensive, repeated generation of large prime numbers from the handshake's critical path.	Converts the single most expensive step of the handshake into a fixed, one-time cost, reducing per-handshake latency substantially.
Session resumption with bounded cache	Returning clients are served from a cache lookup instead of repeating the full asymmetric handshake, with a least-recently-used eviction policy bounding memory use.	Resumed handshakes complete in a small fraction of the time of a full handshake, while the bounded cache prevents unbounded memory growth on busy servers.
Full but efficient certificate validation	Validation checks are ordered so the cheapest checks (subject, expiry) run before the more expensive signature/chain verification, short-circuiting early on obviously invalid certificates.	Reduces average validation cost without weakening the checks performed.

Step 4 — Analysis / Evaluation

Evaluation Criterion	Finding
Latency reduction	Removing per-handshake prime generation and adding session resumption together remove the two most expensive recurring costs from the handshake, so average handshake time drops substantially, particularly for repeat connections.
Correctness restored	Switching to a cryptographically secure random source removes the nonce-collision and predictability issues that were causing intermittent handshake failures.
Security posture	Full chain-of-trust certificate validation closes a serious security gap in the original implementation, where an invalid certificate could otherwise be silently accepted.
Scalability	The bounded session cache allows the system to serve many concurrent clients with fast resumed handshakes without memory usage growing without limit.

Security vs. Performance Trade-off

Reusing a standard, publicly vetted Diffie–Hellman group does not weaken security, since such groups are chosen precisely to be safe for reuse across many systems. Session resumption, however, does trade off some forward secrecy during the resumption window, because a compromised resumption ticket could allow an attacker to resume a session without repeating the full key exchange. This is mitigated by keeping resumption tickets short-lived and evicting them from the cache after a limited time, which balances the latency benefit of resumption against the reduced (but time-bounded) exposure it introduces.

Question 3: Debugging and Optimizing Digital Signature Algorithm (DSA)

Scenario

A Python implementation of DSA generates invalid signatures during verification. The task requires identifying issues in parameter selection and random number generation, fixing signature generation and verification steps, optimizing modular arithmetic operations, and evaluating the impact of randomness quality on signature security.

Why the Symptom Occurs

DSA signature generation depends on a secret per-signature random value, commonly called k . If k is not generated correctly — wrong range, reused across signatures, or not coprime with the group order — the resulting signature can be mathematically invalid, or in the worst case, expose the private signing key. Separately, verification failures can also be caused by an implementation bug in the verification formula itself, independent of whether the signature was validly generated.

Step 1 — Problem Identification

The table below lists each defect found in the original implementation, its underlying root cause, and the concrete impact it has on correctness or security.

Defect Identified	Root Cause	Impact
Reused per-signature secret value	The random secret value k , which must be unique for every signature, is generated once and then reused for every subsequent signing operation.	This is the most severe defect: reusing k across two signatures on different messages allows an attacker to algebraically recover the private signing key from the two signatures alone.
Incorrect range and validity checking for k	The secret value k is drawn from the wrong numeric range and is never re-drawn if it produces a degenerate (zero-valued) signature component.	Occasionally produces signatures with a zero component, which are invalid by definition under the DSA specification, and does not use the specification's mandated retry behaviour.
No coprimality check before computing a modular inverse	The implementation computes a modular inverse of k without first confirming that k and the group order share no common factor.	Can produce an incorrect inverse or a runtime error, silently corrupting the signature's mathematical validity.

Defect Identified	Root Cause	Impact
Incorrect modular reduction order during verification	The verification formula performs the modulo-by-the-large-prime reduction and the modulo-by-the-subgroup-order reduction in the wrong sequence.	Even a validly generated signature is computed as mathematically different from the expected value during verification, causing verification to fail for a perfectly valid signature — directly explaining the reported symptom.

Step 2 — Debugging Approach

Each defect above is resolved using a systematic, targeted correction. The table maps every defect to the specific debugging action taken and the principle or standard it restores compliance with.

Defect Addressed	Debugging Action Taken	Standard / Principle Restored
Reused secret value	Signature generation is refactored so a fresh secret value is drawn independently for every single signing call, with no caching or reuse across calls.	Uniqueness-per-signature requirement of DSA (FIPS 186-4)
Range and validity checking	The secret value is drawn from the correct valid range, and the algorithm re-draws a new value and retries whenever a degenerate signature component would result, exactly as the specification requires.	Retry-on-degenerate-output rule mandated by the DSA specification
Coprimality check	The implementation confirms the secret value shares no common factor with the group order before computing its modular inverse.	Well-defined modular inverse requirement
Modular reduction order	The verification formula is corrected to reduce modulo the large prime first, and only then reduce that result modulo the subgroup order, matching the defined verification equation.	Correct DSA verification equation

Step 3 — Optimization

Once correctness is restored, the implementation is optimized for the scale/performance target stated in the scenario. Each technique below targets the true bottleneck rather than an incidental inefficiency.

Optimization Technique	What It Changes	Performance Benefit
Simultaneous (combined) modular exponentiation	The two modular exponentiations required during verification are combined into a single, shared computation that reuses intermediate squaring steps instead of computing each exponentiation completely independently.	Verification runs substantially faster than performing two full, separate exponentiations, which matters for systems verifying many signatures per second.
Early validity range checks before expensive computation	Signature components are range-checked cheaply before any modular exponentiation is performed.	Invalid signatures are rejected immediately without incurring the cost of the expensive modular arithmetic.

Step 4 — Analysis / Evaluation

Evaluation Criterion	Finding
Randomness quality is foundational, not incidental	The secret per-signature value must be secret, uniformly random, unique per signature, and within the correct range. Any weakness in how it is generated — reuse, predictability, bias, or an incorrect range — can allow an attacker to recover the private signing key, not merely produce an invalid signature.
Real-world relevance	Reused or predictable per-signature secret values have historically led to real private-key compromises in deployed systems with weak random number generators, which is why current standards mandate a fresh, unpredictable value for every signature.
Correctness vs. security bugs	The verification-formula ordering defect is a correctness bug: it causes valid signatures to be wrongly rejected. The secret-value reuse defect is a security bug: it can leak the private key even when every signature looks individually valid. Both had to be fixed, but they represent different classes of risk.
Performance impact of the fix	Adding proper range checks and retry logic adds negligible overhead, since degenerate outputs are rare; the combined-exponentiation optimization more than offsets this by reducing the dominant verification cost.

Question 4: Debugging Key Management System

Scenario

A Python-based key management system fails to securely store and retrieve cryptographic keys. The task requires identifying flaws in key storage (plaintext storage, improper encryption), fixing key lifecycle issues (generation, distribution, rotation), optimizing secure storage using encryption and access control, and analyzing how poor key management can compromise an entire cryptographic system.

Why the Symptom Occurs

A key management system's entire purpose is to protect cryptographic key material at rest and to control who can access it. When keys are stored in plaintext with no encryption, no access control, and no lifecycle policy, the system fails at its core purpose even if the cryptographic algorithms that use those keys are themselves implemented correctly — the weakest link is the storage and governance layer, not the algorithms.

Step 1 — Problem Identification

The table below lists each defect found in the original implementation, its underlying root cause, and the concrete impact it has on correctness or security.

Defect Identified	Root Cause	Impact
Plaintext key storage	Generated and rotated keys are written directly to a storage file with no encryption applied to the key material itself.	Anyone who gains access to the storage file — through misconfiguration, backup exposure, or insider access — immediately obtains every key in the system.
No access control on key retrieval	Any caller can request and receive any key by its identifier, with no authentication or authorization check performed.	There is no way to restrict which parts of a system, or which users, are permitted to access a given key, and no record of who accessed what.
Destructive key rotation	Rotating a key simply overwrites the previous key value in place, with no versioning.	Data previously encrypted under the old key becomes permanently undecryptable once the key is rotated, since the old key value no longer exists anywhere.

Defect Identified	Root Cause	Impact
No key lifecycle metadata	Keys are stored with no creation date, expiry date, or version information attached.	There is no mechanism to enforce periodic rotation or to detect and retire stale keys, so compromised or aging keys can remain in use indefinitely.

Step 2 — Debugging Approach

Each defect above is resolved using a systematic, targeted correction. The table maps every defect to the specific debugging action taken and the principle or standard it restores compliance with.

Defect Addressed	Debugging Action Taken	Standard / Principle Restored
Plaintext key storage	Individual keys are now encrypted under a separate master key before being written to storage, so the stored file never contains usable key material on its own (an envelope-encryption pattern).	Encryption-at-rest for cryptographic key material
No access control	Every key operation is gated behind a role check, and only explicitly authorized roles may generate, retrieve, or rotate a given key, with every access recorded in an audit log.	Least-privilege access control and auditability
Destructive rotation	Rotation now appends a new, versioned key entry rather than overwriting the previous one, so older key versions remain available for decrypting data that was encrypted under them.	Key versioning to preserve decryptability of historical data
Missing lifecycle metadata	Every key version is stored with a creation timestamp and an expiry timestamp, enabling enforcement of a defined rotation policy.	Defined key lifecycle (generation, distribution, expiry, rotation, retirement)

Step 3 — Optimization

Once correctness is restored, the implementation is optimized for the scale/performance target stated in the scenario. Each technique below targets the true bottleneck rather than an incidental inefficiency.

Optimization Technique	What It Changes	Performance Benefit
One-time master-key derivation	The expensive, intentionally-slow derivation of the master key from a passphrase is performed once when the system starts, and the derived key is kept only in memory for reuse, instead of being recomputed on every operation.	Removes a deliberately expensive computation from the hot path of every single key operation, so day-to-day key access remains fast even though the underlying derivation is intentionally slow for security reasons.
Batched storage reads	Storage file reads are consolidated so each key operation reads the storage file once rather than multiple times.	Reduces redundant disk I/O, improving throughput under frequent key access.
Bounded in-memory caching of frequently used keys with a short expiry	Frequently accessed decrypted keys can be held briefly in memory rather than being decrypted fresh on every single request.	Reduces repeated decryption overhead for hot keys while the short expiry limits how long decrypted key material remains exposed in memory.

Step 4 — Analysis / Evaluation

Evaluation Criterion	Finding
Systemic impact of poor key management	Because the original system stored every key in plaintext with no access control, a single storage compromise (misconfigured backup, path traversal vulnerability, insider access) compromises every secret the system was meant to protect. This is categorically worse than compromising a single password, since keys typically protect many other pieces of sensitive data simultaneously.
Blast-radius reduction	With envelope encryption, exfiltrating the storage file alone is not sufficient to recover usable keys, since the attacker would also need the separately-held master key, which is never persisted to disk.
Lifecycle management value	Versioned rotation means a compromised key can be rotated without losing the ability to decrypt data that was encrypted before the rotation, while expiry limits how long any single key remains in active use, reducing the value of a compromised key to an attacker.
Governance and traceability	Role-based access control combined with audit logging converts an ungoverned 'anyone can read any key' system into a traceable, least-privilege system consistent with recognized key-management guidance.

Question 5: Optimization of Secure File Transfer Protocol (SFTP-like System)

Scenario

A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption. The task requires debugging encryption/decryption synchronization, fixing packet handling and integrity verification, optimizing throughput for large file transfers (≥ 100 MB), and comparing the optimized system with insecure file transfer methods.

Why the Symptom Occurs

In encrypted file-transfer systems, data corruption almost always traces back to one of two causes: (1) reuse of an encryption nonce/counter value across multiple packets, which breaks the security and correctness guarantees of stream-style encryption modes, or (2) the absence of a per-packet integrity check, which means a corrupted, reordered, or tampered packet is decrypted and written to disk without ever being detected. Slow throughput separately traces back to inefficient chunking and fully serialized (non-overlapping) I/O and encryption work.

Step 1 — Problem Identification

The table below lists each defect found in the original implementation, its underlying root cause, and the concrete impact it has on correctness or security.

Defect Identified	Root Cause	Impact
Reused encryption nonce across packets	The same fixed nonce/counter value is used to encrypt every packet instead of a fresh, unique value per packet.	For the stream-based encryption mode used, reusing the nonce causes the same keystream to be produced repeatedly, which both leaks information and, combined with any packet loss or reordering, produces corrupted plaintext on decryption.
Fixed-size padding without tracking true length	Short final chunks are padded to a fixed size before encryption, with no record kept of the original, unpadded length.	The receiver cannot distinguish real data from padding, so trailing padding bytes are written directly into the reconstructed file, corrupting its end.
No per-packet integrity verification	Packets are decrypted and written to disk with no authentication tag or integrity check performed beforehand.	A tampered, truncated, or corrupted packet is silently decrypted into garbage and saved, rather than being detected and rejected.

Defect Identified	Root Cause	Impact
Fully serialized read/encrypt/write pipeline	Each packet is read from disk, encrypted, and written out one at a time, with no overlap between disk I/O and computation.	Disk read latency, encryption time, and disk write latency are never overlapped, so total transfer time is close to the sum of all three rather than the maximum of them, explaining the slow throughput.

Step 2 — Debugging Approach

Each defect above is resolved using a systematic, targeted correction. The table maps every defect to the specific debugging action taken and the principle or standard it restores compliance with.

Defect Addressed	Debugging Action Taken	Standard / Principle Restored
Reused nonce	The implementation is switched to generate a fresh, unique nonce for every packet, and to an authenticated encryption mode that binds each packet's sequence number to its ciphertext.	Nonce uniqueness requirement for stream-style / counter-based encryption modes
Fixed-size padding	Chunks are transmitted at their true, natural length rather than being padded to a fixed size, removing the need for padding and any ambiguity about original length.	Length-preserving packet handling
No integrity verification	Every packet now carries an authentication tag that is verified before any decrypted bytes are written to disk; a failed check causes the packet to be rejected rather than silently accepted.	Authenticated encryption (confidentiality plus integrity and authenticity)
Serialized pipeline	The transfer pipeline is refactored so that reading the next chunk from disk can overlap with encrypting the current chunk, instead of waiting for each step to fully complete before starting the next.	Overlapped I/O and computation for reduced end-to-end latency

Step 3 — Optimization

Once correctness is restored, the implementation is optimized for the scale/performance target stated in the scenario. Each technique below targets the true bottleneck rather than an incidental inefficiency.

Optimization Technique	What It Changes	Performance Benefit
Larger chunk size	The size of each transmitted chunk is increased substantially from a very small fixed size to a much larger size.	Reduces the number of individual encryption calls and per-call overhead needed to transfer a given file, which matters most for large (≥ 100 MB) files.
Overlapped I/O and encryption (pipelining)	Disk reads for upcoming chunks are performed concurrently with encryption of the current chunk rather than strictly sequentially.	Total transfer time approaches the larger of the I/O time or the encryption time rather than their sum, since the disk-bound and CPU-bound portions of the work now happen in parallel.
Sequence-ordered reassembly	Received packets are reassembled in strict sequence order using their sequence numbers, rather than assuming packets arrive and are processed in order.	Prevents out-of-order packets from producing a corrupted or misordered output file, improving reliability without adding significant overhead.

Step 4 — Analysis / Evaluation

Evaluation Criterion	Finding
Correctness restored	Switching from a reused nonce to a fresh nonce per packet, combined with mandatory integrity verification before writing to disk, means corrupted or tampered packets are now detected and rejected rather than silently written — directly resolving the reported data-corruption symptom.
File fidelity	Removing fixed-size padding means the reconstructed file is identical, byte for byte, to the original, whereas the earlier version always appended extraneous padding bytes to files whose size did not evenly divide the chunk size.
Throughput improvement	Increasing chunk size and overlapping disk I/O with encryption together substantially reduce wall-clock transfer time for large files, since both the per-call overhead and the serialized-waiting time are reduced.

Evaluation Criterion	Finding
Comparison with insecure transfer methods	An unencrypted transfer method is faster in raw throughput because it performs no cryptographic work at all, but it exposes file contents and any transmitted credentials in plain form on the network and provides no way to detect tampering. The optimized, authenticated encrypted pipeline narrows this performance gap to a modest, mostly I/O-bound overhead while providing confidentiality, integrity, and authenticity that the insecure method entirely lacks.

Security vs. Performance Trade-off

The optimizations applied (larger chunks, overlapped I/O, sequence-based reassembly) do not weaken security in any way — they change only how the work is scheduled and batched, not what is encrypted or how integrity is verified. The meaningful trade-off is against unencrypted transfer methods generally: those remain marginally faster in raw terms but sacrifice confidentiality and tamper detection entirely, which is not an acceptable trade-off for any transfer involving sensitive data.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated

Cod e Quali ty	15%	Produc es clean, well- structure d code with proper comments	Code is mostly clear with minor documentat i on gaps	Code lacks structure and proper documentati o n	Poorly written code with no document ati on
-------------------------	-----	---	---	---	--

		and documentat io n			
--	--	------------------------------	--	--	--